

Commentaire

REMARQUES GENERALES :

- toujours commencer par expliquer en français ce que vous allez faire en Python.
- **Toujours tester sur des exemples !**

Exercice 1

Il y a deux types de boucles : la boucle FOR et la boucle WHILE.

La première peut s'utiliser de deux manières :

- soit en explorant les **valeurs** d'un objet *itérable* (chaîne de caractères, liste, tuple, range, tableau Numpy) : `for element in maliste :`
- soit en explorant les indices de cet objet : `for indice in range(len(machaine)) :`

Dans cet exercice nous voyons différents problèmes où il peut être utile ou nécessaire d'utiliser telle ou telle forme de boucle.

1. a) Que fait la fonction suivante ?

```
def mystere(parametre) :  
    x = parametre[0]  
    for y in parametre[1:] :  
        if y < x :  
            x = y  
    return x
```

b) Faire une fonction `indice_minimum(maliste)` qui renvoie l'indice de l'élément (ou d'un des éléments) de valeur minimale dans `maliste` (ou suppose sans le vérifier que `maliste` est formée de nombres).

2. Un nombre entier est un *palindrome* s'il se lit de la même manière dans les deux sens. Exemples : 2332 ou 56765.

Un entier p est dit *triangulaire* lorsqu'il existe un entier n tel que $p = 1 + 2 + 3 + \dots + (n - 1) + n$.

a) On veut faire une fonction `est_triangulaire(n)` qui prend un entier n et qui renvoie `True` si n est triangulaire et `False` sinon. Complétez-la (y compris les commentaires) :

```
def est_triangulaire(n) :  
    k = 1 # Cette variable .....  
    S = 1 # Cette variable .....  
    while .....  
        .....  
        .....  
    return .....
```

- b) Faire une fonction `est_palindrome(n)` qui prend un entier n et qui renvoie `True` si n est un palindrome et `False` sinon.
Indication : On pourra penser au transtypage `int -> str`.
- c) Faire un script qui donne les 10 premiers entiers qui sont à la fois palindromes et triangulaires.
3. a) Faire une fonction `liste_des_mots(maphrase)` qui renvoie la liste des mots du `str maphrase` (deux mots étant séparés par un ou des espaces)
 Exemple : `liste_des_mots('o bladi, o blada')` doit renvoyer `['o', 'bladi,', 'o', 'blada']`.
- b) Faire une fonction `mot_y_es_tu(maphrase, monmot)` qui renvoie `True` si `monmot` est un mot de `maphrase` (séparé du mot précédent et du mot suivant par un ou des espaces, donc) et `False` sinon.
- c) Faire une fonction `mot_y_es_tu_II(maphrase, monmot)` qui renvoie la liste (éventuellement vide) des indices des occurrences de `monmot` dans `maphrase`.
4. Faire une fonction `dichotomie(f, a, b, epsilon)` qui donne une approximation d'UNE solution de l'équation $f(x) = 0$ sur l'intervalle $[a, b]$ (ou $[b, a]$, peu importe l'ordre). Cette approximation sera donnée à epsilon près.
 Donner un exemple d'application.
5. Les *diviseurs propres* d'un entier n sont les entiers k tels que $1 \leq k < n$ et k divise n .
 Les nombres parfaits sont les nombres supérieurs ou égaux à 2 et égaux à la somme de leurs diviseurs propres. Par exemple $6 = 1 + 2 + 3$.
- a) Faire une fonction `diviseurs_propres(n)` qui renvoie la liste des diviseurs propres de n , sans doublons.
 On pourra varier les plaisirs en faisant deux versions : l'une utilisant une boucle `for` et l'autre une boucle `while`. Laquelle préférez-vous ?
- b) Faire un script qui détermine la liste des 4 premiers « nombres parfaits ».
6. En fin de compte, quand utiliser chaque type de boucles ?

Exercice 2

Quand on ne sait pas par avance quand la boucle doit s'arrêter, on peut décider de faire une boucle `WHILE`, mais on a la possibilité, souvent plus simple de faire une boucle `FOR` qu'on interrompt avec un `break`. Plus élégamment, dans une fonction, il faut se souvenir que le mot clé `return` interrompt immédiatement la fonction (et donc la ou les boucles dans lesquelles il se trouve).

- Lire le paragraphe 2.7 du cours de Python.
- Faire une fonction qui prend une liste de nombre en entrée et qui renvoie `True` s'il y a au moins un zéro parmi eux et `False` sinon.
 - Avec une boucle `while`.
 - Avec une boucle `for` et un `break`.
 - Avec une boucle `for` et un ou des `return` bien placés
- Idem pour une fonction qui prend une liste de nombres en entrée et qui renvoie `True` s'ils sont tous nuls et `False` sinon.